

Gaia: Hybrid Hardware Acceleration for Serverless AI in the 3D Compute Continuum

Maximilian Reisecker
Distributed Systems Group
TU Wien

Vienna, Austria
e11908094@student.tuwien.ac.at

Thomas Pusztai
Distributed Systems Group
TU Wien
Vienna, Austria
t.pusztai@dsg.tuwien.ac.at

Cynthia Marcelino
Distributed Systems Group
TU Wien

Vienna, Austria
c.marcelino@dsg.tuwien.ac.at

Stefan Nastic
Distributed Systems Group
TU Wien
Vienna, Austria
snastic@dsg.tuwien.ac.at

Abstract

Serverless computing offers elastic scaling and pay-per-use execution, making it well-suited for AI workloads. As these workloads run in heterogeneous environments such as the Edge-Cloud-Space 3D Continuum, they often require intensive parallel computation, which GPUs can perform far more efficiently than CPUs. However, current platforms struggle to manage hardware acceleration effectively, as static user-device assignments fail to ensure SLO compliance under varying loads or placements, and one-time dynamic selections often lead to suboptimal or cost-inefficient configurations.

To address these issues, we present Gaia, a GPU-as-a-service model and architecture that makes hardware acceleration a platform concern. Gaia combines (i) a lightweight Execution Mode Identifier that inspects function code at deploy time to emit one of four execution modes, and a Dynamic Function Runtime that continuously re-evaluates user-defined SLOs to promote or demote between CPU- and GPU backends. Our evaluation shows that it seamlessly selects the best hardware acceleration for the workload, reducing end-to-end latency by up to 95%. These results indicate that Gaia enables SLO-aware, cost-efficient acceleration for serverless AI across heterogeneous environments.

CCS Concepts

• **Software and its engineering** → **Cloud computing**; **Message passing**; **Middleware**.

Keywords

Serverless, FaaS, GPU, AI, Edge, Cloud, LEO, 3D Continuum

ACM Reference Format:

Maximilian Reisecker, Cynthia Marcelino, Thomas Pusztai, and Stefan Nastic. 2025. Gaia: Hybrid Hardware Acceleration for Serverless AI in the 3D Compute Continuum. In *IEEE/ACM 12th International Conference*

on Big Data Computing, Applications and Technologies (BDCAT '25), December 01–04, 2025, Nantes, France. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3773276.3774299>

1 Introduction

Serverless computing offers elastic scaling, and a pay-per-use model, making it well-suited for a wide range of applications such as real-time data processing, IoT, and AI inference [20, 28, 56]. However, hardware acceleration in serverless platforms remains challenging. Functions are typically deployed without knowledge of the underlying hardware, while developers must manually decide whether a function should run on a CPU or GPU. The static assignment of functions to hardware resources increases developer complexity and often results in inefficient utilization of heterogeneous environments, where resources may be unavailable for function requirements.

Recently, the rapid growth of Low Earth Orbit (LEO) satellite megaconstellations equipped with onboard computing resources and inter-satellite links (ISLs) has enabled novel paradigms in space, such as serverless computing [7, 31] in the 3D Compute Continuum [22, 39, 45]. The 3D Compute Continuum unifies the Edge, Cloud, and LEO satellites into one system, creating a seamless application flow. In such environments, efficient hardware acceleration becomes critical. In particular, LEO satellites introduce challenges that amplify the problem, such as resource availability as satellites move in and out of connectivity windows, constrained hardware capacity, and network conditions that fluctuate with orbital dynamics [6, 16, 26]. Therefore, a function assigned to a GPU during deployment time may not find GPU capacity on a newly scheduled satellite node, causing scheduling delays or failures. Moreover, one-off dynamic decisions may allocate GPUs unnecessarily during temporary workload peaks, leading to severe cost inefficiencies in an environment where resources are scarce and expensive. As a result, functions may suffer from performance degradation under workload drift or incur increased costs due to overprovisioned GPU usage.

Common approaches that enable serverless hardware acceleration include: (a) *Static* state-of-the-art [30, 41, 42] and commercial [8, 10, 50] approaches require the developer to decide whether to run on a CPU or GPU. While this provides fine-grained control, it



This work is licensed under a Creative Commons Attribution 4.0 International License. BDCAT '25, Nantes, France

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2286-8/25/12

<https://doi.org/10.1145/3773276.3774299>

increases developer complexity and inefficient resource allocation, as a platform in a heterogeneous environment might not have the capacity to execute the required hardware specifications. This leads to delays and potential errors. (b) *Dynamic* approaches migrate to GPUs based on the current function performance [4, 5] or determine it before execution, based on resource availability and user-defined SLOs such as performance, and cost [19, 49]. However, most dynamic approaches make a one-off device choice and do not revisit it as load changes, which can lead to performance degradation under workload drift or increased costs.

Existing state-of-the-art approaches that enable hardware acceleration, i.e., GPU serverless function execution, either rely on static developer choices or one-off dynamic device selection [30, 47]. Therefore, they fail to adapt to heterogeneous and dynamic environments such as the 3D Compute Continuum. As satellites move in and out of orbit and the connectivity windows shift, the platform must continuously reevaluate node capacity and available hardware resources. To address this gap, we present Gaia, an abstraction for seamless hardware acceleration in serverless platforms. Gaia enables the platform to automatically identify whether a function requires GPU or CPU execution and to adjust execution dynamically at runtime, while ensuring SLO compliance. Our contributions include:

- *Gaia* a hybrid acceleration model and architecture that abstracts function requirements and enables seamless runtime adjustment between CPU and GPU, while complying with SLOs.
- *Execution Mode Identifier* that identifies functions with GPU-affine operations at deployment time, providing hints to the serverless platform without requiring developer intervention, such as configuration or code changes.
- *Dynamic Function Runtime Reconfiguration* that dynamically adjusts hardware allocation during execution as workloads or system conditions change, moving task execution to another node if necessary, thus preventing performance degradation and cost inefficiencies inherent to one-off decisions.

This paper has eight sections. Section 2 presents the illustrative scenario and research questions. Section 3 describes the Gaia model as well as the architecture overview. Section 4 describes the static hardware identifier and the dynamic runtime adaptation introduced by Gaia and their usage. Section 5 shows the prototype implementation details. Section 6 discusses the experiments and evaluation, Section 7 presents related work. Section 8 concludes with a final discussion and future work.

2 Illustrative Scenario and Research Challenges

To better understand the challenges associated with hardware acceleration execution, in this section, we present our motivating illustrative scenario and our research challenges.

2.1 Illustrative Scenario

Early detection of deforestation in remote regions requires timely data processing and efficient use of heterogeneous resources across the 3D Continuum. In our deforestation use case (Fig. 1), ground-based sensors and drones collect environmental indicators such as high-resolution imagery, temperature, and CO₂ levels. When in

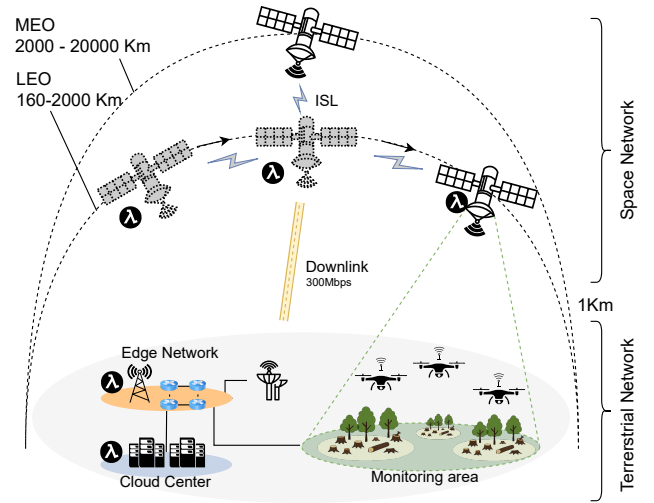


Figure 1: Deforestation Illustrative Scenario with Serverless Functions Execution in the 3D Compute Continuum

range, drones offload data to nearby edge nodes; otherwise, they transmit data directly to LEO satellites, ensuring high availability even when the edge network is unavailable. Moreover, LEO satellites can also fuse ground data from drones with imagery from Earth Observation (EO) satellites, including optical and infrared feeds. The combined streams are processed through a serverless workflow consisting of ingestion, image segmentation, and pattern recognition functions to detect deforestation hotspots. By distributing these tasks across edge, cloud, and space, the 3D Continuum reduces data transfer overheads, lowers detection latency, and supports scalable analysis of daily large EO datasets up to 1.5TB that would otherwise be costly to transmit to ground-based cloud services [3, 17]. Gaia enhances this workflow by abstracting CPU/GPU hardware selection, ensuring that resource-intensive functions, such as Object Detection or Prepare Dataset, seamlessly exploit available GPUs at Edge nodes or onboard LEO satellites, without requiring developers to manually specify execution devices. The Gaia static code analyzer identifies GPU-affine functions at deployment time, providing execution hints that prevent unnecessary GPU allocation for lightweight functions such as Ingest. Furthermore, Gaia's adaptive runtime reconfiguration allows the platform to dynamically reassign functions when satellites move in and out of orbit or when workload intensity changes, maintaining SLO compliance despite fluctuating resource availability. Thus, Gaia enables the deforestation detection workflow to leverage onboard GPUs when ground links are congested, or to fall back to CPUs when GPU capacity is limited or unavailable, optimizing performance and cost while ensuring uninterrupted serverless execution.

2.2 Research Challenges

RC-1: How can serverless AI functions maintain optimal execution performance under resource constraints and dynamic conditions of the 3D Compute Continuum?

The 3D Compute Continuum paradigm unifies Edge-Cloud and LEO satellites by leveraging the unique strengths of each layer. Onboard processing is particularly beneficial for downstream applications such as Earth Observation analytics, as it avoids costly and delay-prone downlinking that depends on orbital passes, typically incurring multiple hours [12, 54]. Downstream applications often involve AI workloads that require hardware acceleration, such as GPUs. As satellites move in and out of range, certain functions must remain Earth-anchored to geo-specific monitoring regions, requiring migration to different satellites and adaptation of execution to maintain SLO compliance. While GPUs are becoming more common on satellites [9, 59], their capacities are still limited by power constraints [13, 37], thermal constraints, and radiation hardening requirements [18, 55, 57]. Thus, it is imperative to use the available capacities wisely.

RC-2: How to seamlessly identify functions' hardware requirements, such as GPU?

Identifying the hardware requirements of a serverless function is an important, yet challenging, task to ensure that its end-to-end runtime SLO is met and that costs are limited [32, 33]. For CPU and memory requirements, there are various approaches to automate this through offline profiling [11, 46, 51, 58, 62] or through online optimization that adjusts function resources based on previous results [15, 29, 38, 61]. For GPUs, serverless functions commonly use either a granular sharing approach, where an entire GPU is allocated to a function [20, 23, 25], or spatial partitioning [14, 24, 60]. However, running kernels cannot be interrupted; hence, allocations persist for their execution duration. To avoid having too few resources, many systems tend to overprovision GPU resources, thus leading to wasted capacities [36], something that must be avoided on satellites or Edge devices.

RC-3: How to dynamically change function runtime between CPU and GPU to balance performance and cost SLOs?

Typically, serverless functions rely on the runtime that bridges the communication between the hardware and the function [21, 52]. Nevertheless, developers typically have to statically select whether the function should use a GPU-enabled runtime (which has a longer cold start delay) or a CPU-only runtime [30, 41, 42]. This may result in both overprovisioning of GPU resources or limiting functions to the CPU, even though they could benefit from a GPU. A simple approach to make this decision dynamic is to run a function on the CPU first and move it to the GPU if it violates the end-to-end SLO [48]. A more complex approach to dynamicity is to predict a function's execution time and to dynamically allocate it to the CPU or GPU depending on the end-to-end SLO and cost constraints [49]. However, estimating the execution time of a function is not trivial, and running this decision logic for every invocation may add considerable overhead.

3 Gaia Design Principles and Architecture

In this section, we outline Gaia's design principles and describe the architecture that implements them.

3.1 Gaia Design Principles

Gaia is designed to make hardware acceleration a platform concern while keeping functions portable and reducing developer effort.

Therefore, we follow the principles that guided our architecture and are reflected in the mechanisms described in Sections 4 and 5.

Hardware-agnostic. Functions are authored without binding to hardware acceleration devices; execution mode is inferred at deploy time and adjusted at runtime.

Zero Developer Friction. No code changes are required; developers can opt into `auto (Gaia)|cpu|gpu` or rely on the execution mode identifier to transparently select the best hardware acceleration.

Intelligent Start & Dynamic Switching. Upon deployment of a function, Gaia performs a static analysis to determine the best-suited initial configuration. This decision is periodically re-evaluated at runtime, causing a dynamic switch to a different execution mode if necessary.

Observability by Design. Each runtime decision is backed by telemetry, which monitors SLO requirements such as latency and throughput, and persists rationale (last-mode, measured latencies).

3.2 Gaia Architecture

Gaia leverages static code features to identify hardware execution mode and monitoring to evaluate dynamic runtime changes for serverless functions across heterogeneous hardware acceleration backends with minimal developer burden. Functions are deployed with an initial execution mode inferred offline by the static code analyzer; at runtime, Gaia continuously evaluates SLO metrics to promote or demote the function's backend, aligning performance and cost with SLOs across the 3D Compute Continuum.

The Gaia architecture, shown in Fig. 2, is organized into three planes: user, control, and data. In the user plane, the Static Code Analyzer classifies functions and annotates the manifest with execution hints. In the control plane, a coordinator performs dynamic runtime management, closing the loop with telemetry to enact safe, seamless mode switches. In the data plane, the function code and runtime execute, interfacing with the underlying hosting stack to utilize proper hardware acceleration, such as CPU or GPU backends, as required.

3.2.1 Gaia Components. Gaia is composed of six main components: Code Analyzer, Build and Deploy, Controllor, Function Runtime Manager, Function Runtime, and Telemetry.

Code Analyzer. The Code Analyzer inspects serverless functions before deployment time. It parses the function code into an abstract syntax tree (AST) to identify GPU-related imports, explicit device calls, and tensor operations. Based on these heuristics, it assigns an execution mode (`cpu`, `cpu_preferred`, `gpu_preferred`, `gpu`) and annotates the function manifest accordingly. This ensures that the serverless platform schedules functions on suitable nodes without requiring changes to user code.

Build and Deploy. Gaia enables three different deployment modes, allowing developers to explicitly specify whether a function should run on CPU, GPU, or use Gaia's adaptive mode. When the adaptive mode is selected, the Code Analyzer is invoked during build and deployment, and its results are embedded in function deployment for the serverless platform.

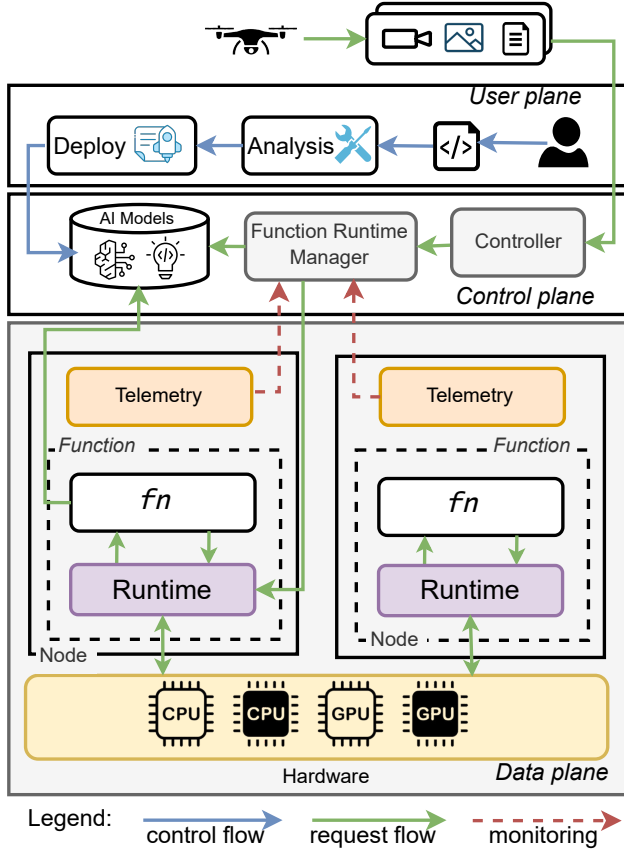


Figure 2: Gaia Architecture

Controller. It manages the lifecycle of functions, and how functions are exposed, scaled, and updated. It manages incoming requests and distributes them across available instances, adjusts the number of active instances based on demand, and orchestrates updates so that newer versions of a function can replace older ones without interrupting ongoing requests. It is also responsible for scheduling functions on nodes with hardware acceleration specification received from the Function Runtime Manager.

Function Runtime Manager. It watches deployed functions and applies function runtime mode switches seamlessly. It interacts with the Controller by giving hardware acceleration hints to dynamically adjust functions when performance and cost SLOs are exceeded. Dynamic shifting between CPU and GPU ensures efficient use of heterogeneous hardware while minimizing disruption to ongoing function executions.

Function Runtime. Gaia supports multiple execution backends via container shims. The default container shim executes functions on CPU, while the NVIDIA container runtime shim enables GPU acceleration. Switching execution mode is achieved by redeploying the function with the appropriate shim, allowing Gaia to leverage GPU resources without modifying user workloads.

Telemetry. It component collects runtime metrics such as request latency, throughput, and resource utilization. These statistics are stored and continuously monitored by the Function Runtime Manager. Telemetry provides the feedback loop that allows Gaia to make informed adaptation decisions, ensuring that execution mode changes are based on actual performance data.

4 Gaia Mechanisms

In this section, we present Gaia mechanisms for adapting serverless functions across CPU and GPU: a static Execution Mode Identifier that infers an initial mode from source code, and a Dynamic Function Runtime that monitors latency and request rate to promote or demote to minimize latency and cost and meet SLOs.

4.1 Execution Mode Identifier

The Execution Mode Identifier provides the first decision step in adapting a function's runtime. Its purpose is to analyze the static code of a serverless function before deployment and assign it to a suitable execution mode. By examining imports, explicit hardware calls, and computational patterns, the component predicts whether the function is best executed on CPU, requires GPU acceleration, or can operate on either with a preference. This static classification reduces unnecessary runtime overhead, ensures clear cases (e.g., explicit GPU usage) are handled deterministically, and leaves more ambiguous cases to be fine-tuned later by the dynamic runtime adaptation mechanism.

The Execution Mode Identifier analyzes the source code of a serverless function to assign one of four execution modes: `cpu`, `gpu`, `cpu_preferred`, and `gpu_preferred`.

The function code is first parsed into an AST (line 1), and flags are initialized to record relevant patterns (line 2). The AST is then traversed node by node (line 3). Imports of deep learning frameworks such as PyTorch or TensorFlow set the `dl_import` flag (line 5). Explicit GPU calls such as `torch.device("cuda")` or `.to("cuda")` set the `gpu_explicit` flag (line 7). Tensor operations are analyzed to estimate workload size; depending on the result, either `big_ops` or `small_ops` is marked (line 9).

At decision time (starting from line 10), the algorithm follows a hierarchical process. Explicit GPU calls enforce the `gpu` mode. Large tensor operations, combined with deep learning imports, lead to `gpu_preferred`. Small tensor operations or imports without heavy usage result in `cpu_preferred`. When no GPU-related activity is detected, the execution mode defaults to `cpu`. The algorithm finally returns the chosen mode together with the reasoning (line 23).

4.2 Dynamic Function Runtime Adaptation

The Dynamic Function Runtime Adaptation continuously monitors deployed serverless functions and adjusts their execution mode between `cpu_preferred` and `gpu_preferred`. To achieve that the Dynamic Function Runtime Adaptation provides adaptive hints to the orchestrator regarding whether a function should execute on CPU or GPU; the orchestrator then handles the scheduling decisions. It relies on request rate and latency metrics collected from the telemetry, together with configurable thresholds, to decide whether a function should switch its execution environment. This ensures

Algorithm 1 Execution Mode Identifier

Require: Function f
Ensure: $m \in \{\text{CPU}, \text{CPU_PREF}, \text{GPU_PREF}, \text{GPU}\}$, reason r

```

1: Parse  $f$  into AST
2: Initialize flags:  $\text{dl\_import} \leftarrow \text{false}$ ,  $\text{gpu\_explicit} \leftarrow \text{false}$ ,
    $\text{big\_ops} \leftarrow \text{false}$ ,  $\text{small\_ops} \leftarrow \text{false}$ 
3: for all AST nodes do
4:   if import of torch or tensorflow then
5:      $\text{dl\_import} \leftarrow \text{true}$ 
6:   else if GPU call like  $\text{to}(\text{"cuda"}), \text{.cuda}(),$ 
      $\text{torch.device}(\text{"cuda"})$  and not  $\text{cuda.is\_available}()$ 
     then
7:      $\text{gpu\_explicit} \leftarrow \text{true}$ 
8:   else if tensor operation then
9:     Estimate tensor size; set  $\text{big\_ops}$  or  $\text{small\_ops}$ 
10:  end if
11: end for
12: if  $\text{gpu\_explicit}$  then
13:    $m \leftarrow \text{GPU}$ ,  $r \leftarrow \text{"explicit GPU usage"}$ 
14: else if  $\text{dl\_import}$  and  $\text{big\_ops}$  then
15:    $m \leftarrow \text{GPU\_PREF}$ ,  $r \leftarrow \text{"large tensor ops"}$ 
16: else if  $\text{dl\_import}$  and  $\text{small\_ops}$  and not  $\text{big\_ops}$  then
17:    $m \leftarrow \text{CPU\_PREF}$ ,  $r \leftarrow \text{"small tensor ops"}$ 
18: else if  $\text{dl\_import}$  then
19:    $m \leftarrow \text{CPU\_PREF}$ ,  $r \leftarrow \text{"imports only"}$ 
20: else
21:    $m \leftarrow \text{CPU}$ ,  $r \leftarrow \text{"no GPU-related activity"}$ 
22: end if
23: return  $(m, r)$ 

```

Algorithm 2 Dynamic Function Runtime

Require: $\text{mode} \in \{\text{CPU_PREF}, \text{GPU_PREF}\}$
Require: request rate, latency, thresholds, and saved CPU/GPU latencies
Ensure: Action: change to CPU, change to GPU, or keep mode

```

1: if  $\text{mode} = \text{CPU\_PREF}$  then
2:   if  $\text{request\_rate} > \text{cold\_start\_mitigation\_threshold}$  then
3:     if  $\text{latency} > \text{threshold}$  or  $(\text{recent\_change and latency}$ 
        $> \text{saved\_gpu\_latency} + \text{gap})$  then
4:       return change to GPU
5:     end if
6:   end if
7: else if  $\text{mode} = \text{GPU\_PREF}$  then
8:   if  $\text{request\_rate} > \text{cold\_start\_mitigation\_threshold}$  and  $\text{re-$ 
      $\text{cent\_change and (latency + gap} > \text{saved\_cpu\_latency)}$  then
9:     return change to CPU
10:  end if
11:  if  $\text{request\_rate} < \text{threshold}$  and  $(\text{saved\_cpu\_latency} = \text{None}$ 
     or  $\text{saved\_cpu\_latency} < \text{threshold})$  then
12:    return change to CPU
13:  end if
14: end if
15: return keep mode (default)

```

performance compared to the saved GPU latency plus a safety margin (line 3) triggers a change to the GPU (line 4). If the function runs in *gpu-preferred* mode (line 7), the reevaluator considers switching back to the CPU if two conditions are met. First, when the request rate is high, but the observed latency plus a margin is still worse than the saved CPU latency (line 8), the mode is switched to CPU (line 9). Second, when the request rate falls below a lower threshold and CPU performance is acceptable (line 11), the function is also switched to CPU (line 12). If none of these conditions apply, the function keeps its current mode (line 15).

Cold Start Mitigation. To prevent mode switches caused by cold start artifacts, Gaia incorporates a request-rate safeguard. Mode changes are only considered if the observed request rate exceeds a configurable threshold, ensuring that outlier latencies from rarely invoked functions do not bias decisions. Additional safeguards, such as a performance-gap margin between CPU and GPU latencies, prevent unnecessary oscillations between modes.

5 Prototype Implementation

The prototype of Gaia is available as open-source¹. Gaia dynamic runtime adaptation is written in Python, while the static code analyzer is written in Go.

Execution Mode Identifier. The static analyzer inspects Python functions at deployment time. It parses the function source into an abstract syntax tree (AST) and evaluates indicators such as framework imports (e.g., PyTorch, TensorFlow), explicit GPU calls, and tensor operations. To integrate Gaia classification into the deployment pipeline, the Knative func CLI was extended with a new `-deployment-mode` flag (`auto`, `cpu`, `gpu`). When set to `auto`,

¹<https://github.com/polaris-slo-cloud/gaia>

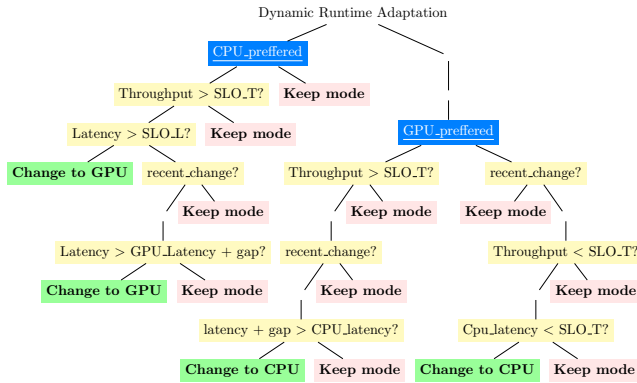


Figure 3: Gaia Dynamic Runtime Function Adaptation

that functions benefit from GPU acceleration when workloads justify it, while avoiding unnecessary GPU usage when the CPU is sufficient.

The function runtime adaptation is shown in Fig. 3 and described Algorithm 2 as pseudocode. The algorithm starts by checking the current execution mode (line 1). If the function runs in *cpu-preferred* mode, a switch to GPU is only considered when the request rate exceeds the cold start mitigation threshold (line 2). Within this condition, high latency or a recent change with worse

the static analyzer is invoked automatically, and its decision is embedded in the Knative Service manifest via annotations and resource limits. This ensures the Kubernetes scheduler correctly places functions on CPU or GPU nodes.

Dynamic Function Runtime Manager. The dynamic function runtime manager is implemented as a Python service running as a Kubernetes Deployment. It uses the Kubernetes API to inspect Knative Service specifications and queries the telemetry service, i.e., Prometheus, to retrieve runtime metrics. When defined conditions are met, the manager updates the Knative Service manifest to patch resource annotations, allowing Knative to roll out a new revision on the appropriate hardware seamlessly.

6 Evaluation

To evaluate Gaia, we designed a set of workloads representing common and contrasting workload types. We aim to evaluate our framework across different execution modes (CPU vs. GPU) and usage scenarios, highlighting strengths and limitations. Each workload was deployed and executed sequentially to ensure isolated measurements. Our goal is to identify the best execution mode for workloads with different characteristics by balancing performance and cost tradeoffs.

Experimental Workloads. We evaluate the framework with four representative workloads that stress different aspects of CPU/GPU execution. Our workloads are: (1) Matrix multiplication models, an intensive computing task where complexity grows with input size, exposing the decision runtime’s ability to adapt execution modes. (2) Image classification with ResNet-18 [27], and (3) Large language model (LLM) inference with TinyLlama [2] represents realistic AI tasks that benefit from GPU acceleration, ranging from moderate to highly demanding inference. (4) Idle wait function, parameterized by `wait_time`, provides a degenerate case with no useful computation, serving as a lower bound for performance comparison. Although our illustrative scenario (Section 2.1) highlights deforestation within the 3D Compute Continuum, Gaia focuses on hardware acceleration across varying workloads, independent of specific application domains. Therefore, to evaluate Gaia, we utilize controlled and representative AI workloads, including compute-intensive tasks, small AI models, and Large Language Models (LLMs), which reflect the computational and memory characteristics of typical real-world use cases, including the scenarios we illustrate. Thus, our experimental design isolates Gaia’s contributions and facilitates a reproducible, hardware-agnostic evaluation of its decision-making logic.

Metrics and Baselines. For all experimental workloads, we collect latency and Costs. The results are collected for CPU, GPU, and Gaia, and the prices are based on Microsoft Azure pricing model [1]. Our goal is to evaluate for each use case the performance and cost trade-offs of executing on CPU and GPU.

6.1 Experimental Setup

To evaluate Gaia, we deployed a MicroK8s [35] cluster with three nodes with different hardware configurations, to mimic the heterogeneous environments of the 3D Compute Continuum. One node with 4vCPUs, no GPU, and 8GB RAM, and two worker nodes. The

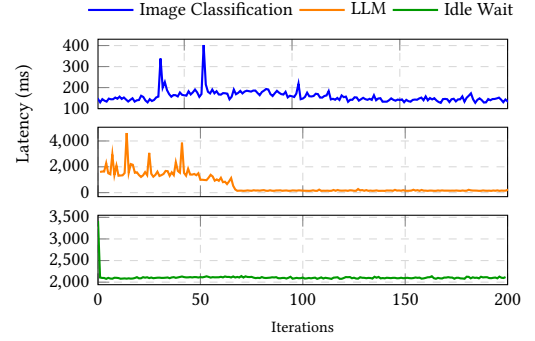


Figure 4: Serverless Function Execution Latency with Dynamic Function Runtime Reconfiguration

first worker node has 8vCPUs, 32 GB RAM, and no GPU, and a worker with 16GB, 64 GB RAM, and a GPU NVIDIA GeForce RTX 3090 24GB. Each node runs Ubuntu Server for ARM 24.04 LTS. The experimental workflow is deployed as Knative [44] services. To avoid bias in the results, all experiments were executed 5 times, and the results shown are the corresponding mean values.

6.2 Experimental Results

6.2.1 Overall latency. Fig. 4 shows that Gaia reduces latency up to 95% once it promotes the backend from CPU to GPU.

Image Classification: Gaia sustains a median response time of 145 ms with rare spikes up to 403ms. However, the latency spikes are not sufficient, and Image Classification runs entirely on CPU.

LLM: In this experiment, Gaia shows the characteristic two-regime curve: an initial steady response times predominantly in 1.3–2.3s (occasional outliers up to 4.6s), followed by a sharp step-down to a stable band around 140–200 ms. Once SLOs are violated, Gaia’s runtime policy switches from a CPU-bound path to an accelerated path (GPU or equivalent fast tier), thereby significantly reducing latency.

Idle Wait: Gaia first changes the execution mode to GPU because of the high latency. After that, the algorithm detects no significant difference latency and thus changes the execution mode back to CPU, maintaining a latency of near 2 seconds 2s.

6.2.2 Matrix Multiplication Results. Fig. 5 shows that with increasing matrix size, Gaia execution mode identifier returns CPU, which explains the initial high latency for GaiaC before the promotion to GPU rises.

Latency: shows that all modes behave similarly for small matrices. After the SLO is reached, Gaia switches to a GPU-back, achieving a clear step-down in response time compared to the CPU. The boxplot in Fig. 5a captures this distributional shift: CPU develops long tails as size increases, while Gaia collapses latency after promotion.

Cost: As execution time grows with matrix size, CPU becomes the most expensive option (Fig. 5b). Gaia tracks CPU-like costs early on, then, after promotion, follows the lower GPU cost curve because requests are completed faster. GPU bears the largest cost in the beginning due to cold-start cost, but is competitive once warmed.

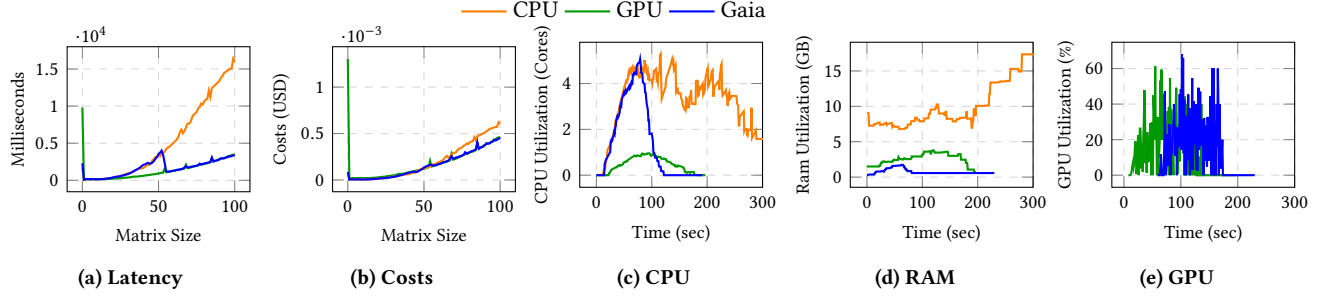


Figure 5: Matrix multiplication: Gaia promotes to GPU once the SLO is hit, producing a clear step-down in latency and cost relative to CPU.

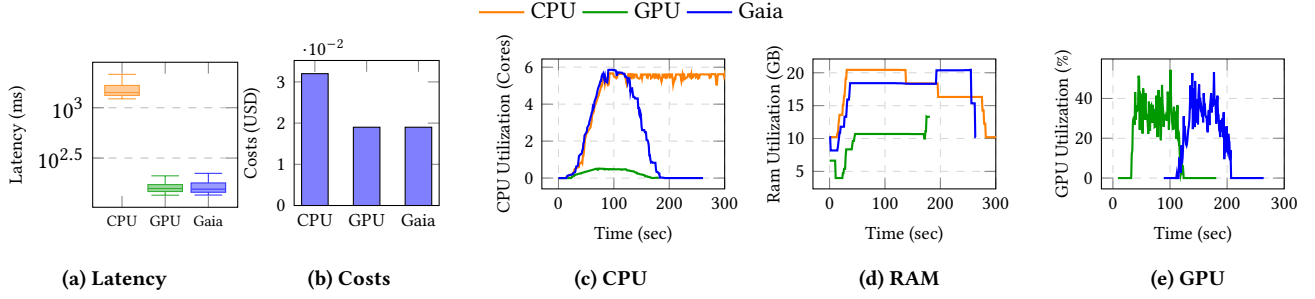


Figure 6: LLM: The initial CPU phase triggers promotion; latency stabilizes at the accelerated tier at a lower cost after switching to GPU

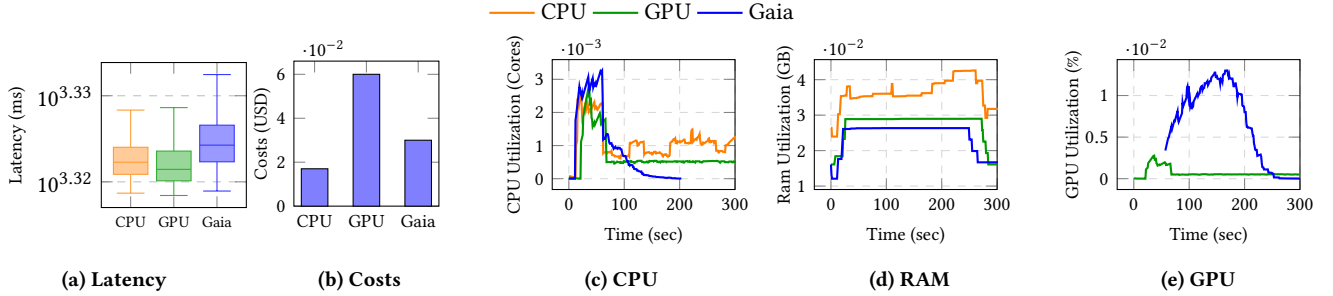


Figure 7: Idle Wait: after GPU promotion and no latency reduction, Gaia downgrades to CPU again.

CPU/RAM usage: Fig. 5c and Fig. 5d show that GPU-backed execution uses significantly less CPU and RAM than the CPU function for the same workload window. The two orange vertical markers denote the Gaia switch points. Fig. 5e shows that Gaia engages the GPU later than GPU-mode: the latter runs on the GPU immediately (after its large cold start), while Gaia promotes only when SLOs are violated, minimizing time spent on expensive or unnecessary acceleration.

6.2.3 LLM Inference Results. For the prompt (“What is the capital of France?”), Gaia identified CPU initially; as soon as the running latency violated the SLO, Gaia promoted to GPU. Post-promotion, Gaia behaves indistinguishably from GPU mode in both latency and cost, while avoiding the large GPU cold-start penalty.

Latency: The boxplot in Fig. 6a shows that once promoted, Gaia aligns with the GPU, while the remaining outliers are attributable to the pre-promotion CPU phase, and CPU exhibits higher latency overall.

Cost: Fig. 6b shows that using the measured totals (CPU: 0.03206, GPU: 0.01914, Gaia: 0.01910), Gaia and GPU are 40% cheaper than CPU. Gaia and GPU are identical following the direct latency collapse after promotion (Fig. 6a).

Resource usage: On CPU, utilization is high; on Gaia after promotion to GPU, CPU load drops and GPU utilization rises to serve inference efficiently (Figs. 6c and 6e). RAM (Fig. 6d) per request stabilizes/declines due to shorter service time on the GPU.

6.2.4 Idle Function Results. Fig. 7 shows that Gaia briefly promotes to the GPU on high latency, finds no improvement, and returns to the CPU. Net effect: CPU-like latency and cost with a short GPU detour.

Latency: High initial latency triggers a promotion to GPU, but because the task is not GPU-amenable, latency does not drop; thus, Gaia demotes back to CPU (Fig. 7a). The boxplot confirms Gaia aligns with CPU, aside from the short GPU window.

Cost: The cost curve shows a small hump exactly during the GPU interval, then converges to the CPU cost trajectory (Fig. 7b).

Resource Usage: Resource traces (Figs. 7c and 7d) remain similar across modes; no sustained CPU or RAM relief appears on the GPU because the workload spends time waiting rather than computing.

7 Related Work

In this section, we present an analysis of the state-of-the-art challenges and how they differentiate from Gaia in the 3D Compute Continuum and GPU-enabled serverless computing.

7.1 Serverless in the 3D Compute Continuum

HyperDrive [45] proposes an architecture for serverless computing in the 3D Continuum, featuring a function scheduler that accounts for the dynamic and heterogeneous characteristics of the environment. While HyperDrive incorporates environment characteristics such as node vicinity, thermal conditions, and power constraints, it does not adapt during runtime to workload-specific characteristics. OrbitFaaS [53] introduces a serverless platform for EO data processing by leveraging in-situ computing capacity. OrbitFaaS introduces orbital models and communication abstractions that enable optimized data processing and AI workloads in the 3D Continuum. Komet [43] proposes a LEO-tailored serverless platform that seamlessly replicates function data in the satellite orbit trajectory. Databelt [40] enables stateful serverless function execution by proactively placing function state along the path of the workflow execution. However, OrbitFaaS, Komet, and Databelt focus on the data management and exchange between functions and do not provide workload-aware execution, which impacts the performance of the function. ML-SDN [34] proposes a serverless software-defined networking (SDN) architecture that dynamically orchestrates communications and computation resources to meet diverse 6G service-level agreements (SLAs). By dynamically adjusting workloads, ML-SDN improves end-to-end learning performance. Krios [7] introduces scheduling abstractions for serverless functions that enable function placement in a specific geographic region. Krios software-defined LEO zones leverage satellite path prediction models to proactively predict serverless functions handovers. However, ML-SDN and Krios primarily focus on network and control-plane orchestration and do not provide function abstraction that allows the platform to seamlessly decide on hardware acceleration. Cosmos [39] proposes a performance-cost tradeoff model for function placement in the Edge-Cloud-Space 3D Continuum. However, Cosmos does not address specific node requirements.

Although the discussed state-of-the-art enables serverless execution in the 3D Compute Continuum, they either focus on network or specific orchestration components that do not consider heterogeneous workload characteristics of serverless functions. In contrast,

Gaia seamlessly identifies hardware acceleration and introduces runtime workload adaptation to meet node and user-defined performance requirements.

7.2 GPU-enabled Serverless Computing

Several approaches enable GPU support in serverless platforms. Kim et al. [30], the OSCAR Extension [41], and KaaS [42] require developers to explicitly specify whether a function executes on a CPU or GPU. Several commercial platforms, including RunPod [50], Cerebrum [8], and Beam Cloud [10], offer GPU sharing but do not include automated decision procedures for CPU versus GPU execution. While this method provides full control over resource allocation and ensures efficient GPU use when configured properly, it increases development complexity and risks inefficient resource utilization. Dynamic approaches such as Llama [49] and DGSF [19] automatically determine the execution device before runtime based on resource availability or user-defined SLOs such as performance, cost. While they allow efficient resource usage and adaptability to workload changes, they also introduce additional decision-making overhead and increase system complexity. Protean [5] proposes a GPU-enabled serverless framework that leverages new-generation NVIDIA GPU capabilities, such as Multi-Instance GPU (MIG) and Multi-Process Service (MPS), to improve SLO compliance and reduce costs. Although Protean avoids CPU-related overhead and maximizes GPU performance, it runs all tasks exclusively on GPUs, sacrificing flexibility and may incur unnecessary costs when workloads do not require GPU acceleration.

Although existing work advances GPU-enabled serverless computing, it relies on manual developer intervention or performs static device selection prior to execution. In contrast, Gaia combines pre-execution decisions with runtime workload adaptation, enabling the platform to seamlessly adjust hardware acceleration during execution according to performance and cost requirements.

8 Conclusion

In this paper, we presented *Gaia*, a GPU-as-a-Service model and architecture that shifts device selection from developers to the platform. Gaia combines an Execution Mode Identifier, which assigns an initial hardware acceleration mode from source code, and a Dynamic Function Runtime that revisits this choice based on SLOs. Our experimental results show that Gaia promotes accelerable workloads and demotes non-accelerable ones. Across matrix multiplication, Image classification, LLM inference, and an idle baseline, reducing end-to-end latency by up to 95%

In future work, we will enhance dynamic runtime decisions by incorporating predictive, learning-based policies and workflow-level objectives. Additionally, we will integrate the dynamics of the 3D Compute Continuum, including LEO handovers, power and thermal constraints, and intermittent links, into our placement and mode decisions.

Acknowledgments

This work has received funding from the Austrian Internet Stiftung under the NetIdee project LEO Trek (ID 7442). Partly funded by EU Horizon Europe under GA no. 101070186 (TEADAL) and GA no. 101192912 (NexaSphere).

References

- [1] 2025. *Azure Container Apps Pricing*. <https://azure.microsoft.com/en-us/pricing/details/container-apps/>. Accessed: 2025-08-13.
- [2] 2025. *TinyLlama*. <https://huggingface.co/TinyLlama/TinyLlama-1.1B-Chat-v1.0>. Accessed: 2025-08-13.
- [3] Airbus. 2024. Airbus built Sentinel-2C satellite successfully launched. <https://www.airbus.com/en/newsroom/press-releases/2024-09-airbus-built-sentinel-2c-satellite-successfully-launched>.
- [4] AWS Batch. [n. d.]. Use a GPU workload AMI — AWS Batch User Guide. <https://docs.aws.amazon.com/batch/latest/userguide/batch-gpu-ami.html>. Accessed: 2025-09-02.
- [5] Vivek M. Bhasi, Aakash Sharma, Rishabh Jain, Jashwant Raj Gunasekaran, Ashutosh Pattnaik, Mahmut Taylan Kandemir, and Chita Das. 2024. Towards SLO-Compliant and Cost-Effective Serverless Computing on Emerging GPU Architectures. In *Proceedings of the 25th International Middleware Conference (MIDDLEWARE '24)*. Association for Computing Machinery, New York, NY, USA, 211–224. <https://doi.org/10.1145/3652892.3700760>
- [6] Debopam Bhattacharjee and Ankit Singla. 2019. Network topology design at 27,000 km/hour. In *Proceedings of the 15th International Conference on Emerging Networking Experiments And Technologies (Orlando, Florida) (CoNEXT '19)*. 341–354. <https://doi.org/10.1145/3359989.3365407>
- [7] Vaibhav Bhosale, Ada Gavrilovska, and Ketan Bhardwaj. 2024. Krios: Scheduling Abstractions and Mechanisms for Enabling a LEO Compute Cloud. In *Proceedings of the 2024 ACM Symposium on Cloud Computing (Redmond, WA, USA) (SoCC '24)*. Association for Computing Machinery, New York, NY, USA, 322–340. <https://doi.org/10.1145/3698038.3698566>
- [8] Cerebrum. [n. d.]. Using GPUs. <https://docs.cerebrum.ai/cerebrum/hardware/using-gpus>
- [9] Roberto Ciardi, Gianluca Giuffrida, Gionata Benelli, Christian Cardenio, and Riccardo Maderna. 2023. GPU@SAT: A General-Purpose Programmable Accelerator for on Board Data Processing and Satellite Autonomy. In *The Use of Artificial Intelligence for Space Applications*, Cosimo Ieracitano, Nadia Mammone, Marco Di Clemente, Mufti Mahmud, Roberto Furfaro, and Francesco Carlo Morabito (Eds.). Springer Nature Switzerland, Cham, 35–47. https://doi.org/10.1007/978-3-031-25755-1_3
- [10] Beam Cloud. 2024. beam-cloud/beta9. <https://github.com/beam-cloud/beta9>
- [11] Robert Cordingley, Sonia Xu, and Wes Lloyd. 2022. Function Memory Optimization for Heterogeneous Serverless Platforms with CPU Time Accounting. In *2022 IEEE International Conference on Cloud Engineering (IC2E)*. 104–115. <https://doi.org/10.1109/IC2E55432.2022.00019>
- [12] CREODIAS. 2025. Timeliness and frequency of Sentinel products explained. <https://creodias.eu/cases/timeliness-and-frequency-of-sentinel-satellite-products-explained/>
- [13] Franco Davoli, Charilaos Kourogiorgas, Mario Marchese, Athanasios Panagopoulos, and Fabio Patrone. 2018. Small satellites and CubeSats: Survey of structures, architectures, and protocols. *International Journal of Satellite Communications and Networking* 37, 4 (2018), 343–359. <https://doi.org/10.1002/sat.1277>
- [14] Aditya Dhakal, Sameer G Kulkarni, and K. K. Ramakrishnan. 2020. GSLICE: controlled spatial sharing of GPUs for a scalable inference platform. In *Proceedings of the 11th ACM Symposium on Cloud Computing (SoCC '20)*. Association for Computing Machinery, 492–506. <https://doi.org/10.1145/3419111.3421284>
- [15] Simon Eismann, Long Bui, Johannes Grohmann, Cristina Abad, Nikolas Herbst, and Samuel Kounnev. 2021. Sizeless: Predicting the Optimal Size of Serverless Functions. In *Proceedings of the 22nd International Middleware Conference (Middleware '21)*. Association for Computing Machinery, 248–259. <https://doi.org/10.1145/3464298.3493398>
- [16] European Space Agency (ESA). [n. d.]. European Data Relay Satellite System (EDRS) Overview. <https://connectivity.esa.int/european-data-relay-satellite-system-edrs-overview>.
- [17] European Space Agency (ESA). [n. d.]. Sentinel-2 operations. https://www.esa.int/enabling_support/operations/sentinel-2_operations.
- [18] Miria M. Finckenor and Kim K. de Groh. [n. d.]. A Researcher's Guide to: Space Environmental Effects. <https://www.nasa.gov/science-research/for-researchers/a-researchers-guide-to-space-environmental-effects/>
- [19] Henrique Fingler, Zhiting Zhu, Esther Yoon, Zhipeng Jia, Emmett Witchel, and Christopher J. Rossbach. 2022. DGSF: Disaggregated GPUs for Serverless Functions. In *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 739–750. <https://doi.org/10.1109/IPDPS53621.2022.00077>
- [20] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 135–153.
- [21] Rastko Gajani, Cynthia Marcelino, and Stefan Nastic. 2025. Performance Isolation for Serverless Functions. *IEEE Transactions on Services Computing* (2025), 1–18. <https://doi.org/10.1109/TSC.2025.3619151>
- [22] Milos Gravara, Andrija Stanisic, and Stefan Nastic. 2025. A Novel Compound AI Model for 6G Networks in 3D Continuum. *arXiv preprint arXiv:2505.15821* (2025).
- [23] Diandian Gu, Yihao Zhao, Yinmin Zhong, Yifan Xiong, Zhenhua Han, Peng Cheng, Fan Yang, Gang Huang, Xin Jin, and Xuanzhe Liu. 2023. ElasticFlow: An Elastic Serverless Training Platform for Distributed Deep Learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (Vancouver, BC, Canada) (ASPLOS 2023)*. Association for Computing Machinery, 266–280. <https://doi.org/10.1145/3575693.3575721>
- [24] Jianfeng Gu, Yichao Zhu, Puxuan Wang, Mohak Chadha, and Michael Gerndt. 2023. FaST-GShare: Enabling Efficient Spatio-Temporal GPU Sharing in Serverless Computing for Deep Learning Inference. In *Proceedings of the 52nd International Conference on Parallel Processing (Salt Lake City, UT, USA) (ICPP '23)*. Association for Computing Machinery, New York, NY, USA, 635–644. <https://doi.org/10.1145/3605573.3605638>
- [25] Runsheng Guo, Victor Guo, Antonio Kim, Josh Hildred, and Khuzaima Daudjee. 2022. Hydrozoa: Dynamic Hybrid-Parallel DNN Training on Serverless Containers. In *Proceedings of Machine Learning and Systems*, D. Marculescu, Y. Chi, and C. Wu (Eds.), Vol. 4. 779–794. https://proceedings.mlsys.org/paper_files/paper/2022/file/0a5209deb79dc584d8ddb41a792d8549-Paper.pdf
- [26] Mark Handley. 2018. Delay is Not an Option: Low Latency Routing in Space. In *Proceedings of the 17th ACM Workshop on Hot Topics in Networks (Redmond, WA, USA) (HotNets '18)*. 85–91. <https://doi.org/10.1145/3286062.3286075>
- [27] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Deep Residual Learning for Image Recognition. <https://doi.org/10.48550/arXiv.1512.03385> [cs].
- [28] Zicong Hong, Jian Lin, Song Guo, Sifu Luo, Wuhui Chen, Roger Wattenhofer, and Yue Yu. 2024. Optimus: Warming Serverless ML Inference via Inter-Function Model Transformation. In *Proceedings of the Nineteenth European Conference on Computer Systems (Athens, Greece) (EuroSys '24)*. 1039–1053. <https://doi.org/10.1145/3627703.3629567>
- [29] Jananie Jarachanthan, Li Chen, Fei Xu, and Bo Li. 2021. Astra: Autonomous Serverless Analytics with Cost-Efficiency and QoS-Awareness. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 756–765. <https://doi.org/10.1109/IPDPS49936.2021.00085>
- [30] Jaewook Kim, Tae Joon Jun, Daeyoun Kang, Dohyeon Kim, and Daeyoung Kim. 2018. GPU Enabled Serverless Computing Framework. In *2018 26th Euromicro International Conference on Parallel, Distributed and Network-based Processing (PDP)*. 533–540. <https://doi.org/10.1109/PDP2018.2018.00090>
- [31] Demi Lei and Ahmed Saeed. 2024. Do We Need a Million Satellites in Orbit? Constellation-as-a-Service with Modular Satellites: Challenges and Opportunities. In *Proceedings of the 2nd International Workshop on LEO Networking and Communication (Washington, DC, USA) (LEO-NET '24)*. 61–66. <https://doi.org/10.1145/3697253.3697262>
- [32] Yongkang Li, Yanying Lin, Yang Wang, Kejiang Ye, and Chengzhong Xu. 2023. Serverless Computing: State-of-the-Art, Challenges and Opportunities. *IEEE Transactions on Services Computing* 16, 2 (2023), 1522–1539. <https://doi.org/10.1109/TSC.2022.3166553>
- [33] Zijun Li, Linsong Guo, Jiagan Cheng, Quan Chen, BingSheng He, and Minyi Guo. 2022. The Serverless Computing Survey: A Technical Primer for Design Architecture. *ACM Comput. Surv.* 54, 10s (2022), 1–34. <https://doi.org/10.1145/3508360>
- [34] Shih-Chun Lin, Chia-Hung Lin, Liang C. Chu, and Shao-Yu Lien. 2023. Enabling Resilient Access Equality for 6G LEO Satellite Swarm Networks. *IEEE Internet of Things Magazine* 6, 3 (2023), 38–43. <https://doi.org/10.1109/IOTM.001.2200272>
- [35] Canonical Ltd. 2024. MicroK8s - Zero-ops Kubernetes for developers, edge and IoT. <https://microk8s.io>. Accessed: 2024-11-11.
- [36] Cunchi Lv, Xiao Shi, Zhengyu Lei, Jinyue Huang, Wenting Tan, Xiaohui Zheng, and Xiaofang Zhao. 2025. Dilu: Enabling GPU Resourcing-on-Demand for Serverless DL Serving via Introspective Elasticity. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (Rotterdam, Netherlands) (ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 311–325. <https://doi.org/10.1145/3669940.3707251>
- [37] Xiao Ma, Mengwei Xu, Qing Li, Yuanzhe Li, Ao Zhou, and Shangguang Wang. 2024. Visions of Edge Computing in 6G. In *5G Edge Computing: Technologies, Applications and Future Visions*. Springer Nature Singapore, Singapore, 179–202. https://doi.org/10.1007/978-981-97-0213-8_9
- [38] Ashraf Mahgoub, Edgardo Barsallo Yi, Karthick Shankar, Sameh Elnikety, Somali Chaterji, and Saurabh Bagchi. 2022. ORION and the Three Rights: Sizing, Bundling, and Prewarming for Serverless DAGs. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, 303–320. <https://www.usenix.org/conference/osdi22/presentation/mahgoub>
- [39] Cynthia Marcelino, Sebastian Gollhofer-Berger, Thomas Pusztai, and Stefan Nastic. 2025. Cosmos: A Cost Model for Serverless Workflows in the 3D Compute Continuum. In *2025 IEEE International Conference on Smart Computing (SMART-COMP)*. 106–113. <https://doi.org/10.1109/SMARTCOMP65954.2025.00072>
- [40] Cynthia Marcelino, Leonard Guelmino, Thomas Pusztai, and Stefan Nastic. 2025. Databelt: A Continuous Data Path for Serverless Workflows in the 3D Compute

- Continuum. *Journal of Systems Architecture* (2025).
- [41] Diana M. Naranjo, Sebastián Risco, Carlos de Alfonso, Alfonso Pérez, Ignacio Blanquer, and Germán Moltó. 2020. Accelerated serverless computing based on GPU virtualization. *J. Parallel and Distrib. Comput.* 139 (May 2020), 32–42. <https://doi.org/10.1016/j.jpdc.2020.01.004>
 - [42] Nathan Pemberton, Anton Zabreyko, Zhoujie Ding, Randy Katz, and Joseph Gonzalez. 2022. Kernel-as-a-Service: A Serverless Interface to GPUs. <https://doi.org/10.48550/arXiv.2212.08146> arXiv:2212.08146 [cs].
 - [43] Tobias Pfandzelter and David Bermbach. 2024. Komet: A Serverless Platform for Low-Earth Orbit Edge Services. In *Proceedings of the 2024 ACM Symposium on Cloud Computing (SoCC '24)*. Association for Computing Machinery, New York, NY, USA, 866–882. <https://doi.org/10.1145/3698038.3698517>
 - [44] Knative Project. 2024. Knative - Kubernetes-based Platform to Build, Deploy, and Manage Modern Serverless Workloads. <https://knative.dev> Accessed: 2024-11-11.
 - [45] Thomas Pusztai, Cynthia Marcelino, and Stefan Nastic. 2024. HyperDrive: Scheduling Serverless Functions in the Edge-Cloud-Space 3D Continuum. In *2024 IEEE/ACM Symposium on Edge Computing (SEC)*. 265–278. <https://doi.org/10.1109/SEC62691.2024.00028>
 - [46] Thomas Pusztai and Stefan Nastic. 2025. ChunkFunc: Dynamic SLO-aware Configuration of Serverless Functions. *IEEE Transactions on Parallel and Distributed Systems* (2025). <https://doi.org/10.1109/TPDS.2025.3559021>
 - [47] Thomas Rausch, Alexander Rashed, and Schahram Dustdar. 2021. Optimized container scheduling for data-intensive serverless edge computing. *Future Generation Computer Systems* 114 (2021), 259–271. <https://doi.org/10.1016/j.future.2020.07.017>
 - [48] Sebastián Risco and Germán Moltó. 2021. GPU-Enabled Serverless Workflows for Efficient Multimedia Processing. *Applied Sciences* 11, 4 (2021). <https://doi.org/10.3390/app11041438>
 - [49] Francisco Romero, Mark Zhao, Neeraja J. Yadwadkar, and Christos Kozyrakis. 2021. Llama: A Heterogeneous & Serverless Framework for Auto-Tuning Video Analytics Pipelines. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC '21)*. Association for Computing Machinery, New York, NY, USA, 1–17. <https://doi.org/10.1145/3472883.3486972>
 - [50] RunPod. [n. d.]. Serverless GPU Endpoints for AI Inference. <https://www.runpod.io/serverless-gpu>
 - [51] Gor Safaryan, Anshul Jindal, Mohak Chadha, and Michael Gerndt. 2022. SLAM: SLO-Aware Memory Optimization for Serverless Applications. In *2022 IEEE 15th International Conference on Cloud Computing (CLOUD)*. 30–39. <https://doi.org/10.1109/CLOUD55607.2022.00019>
 - [52] Simon Shillaker and Peter Pietzuch. 2020. Faasm: Lightweight Isolation for Efficient Stateful Serverless Computing. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, 419–433. <https://www.usenix.org/conference/atc20/presentation/shillaker>
 - [53] Catherine Torres-Charles, Dante D. Sanchez-Gallegos, Diana Carrizales-Espinoza, J.L. Gonzalez-Compean, and Jesus Carretero. 2026. OrbiFaaS: An orbital method to build Continuum Earth Observation Systems. *Future Generation Computer Systems* 174 (2026), 107992. <https://doi.org/10.1016/j.future.2025.107992>
 - [54] Deepak Vasisht, Jayanth Shenoy, and Ranveer Chandra. 2021. L2D2: low latency distributed downlink for LEO satellites. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference (SIGCOMM '21)*. Association for Computing Machinery, New York, NY, USA, 151–164. <https://doi.org/10.1145/3452296.3472932>
 - [55] Chao Wang, Yiran Zhang, Qing Li, Ao Zhou, and Shangguang Wang. 2023. Satellite Computing: A Case Study of Cloud-Native Satellites. In *2023 IEEE International Conference on Edge Computing and Communications (EDGE)*. 262–270. <https://doi.org/10.1109/EDGE60047.2023.00048>
 - [56] Li Wang, Yankai Jiang, and Ningfang Mi. 2024. Advancing Serverless Computing for Scalable AI Model Inference: Challenges and Opportunities. In *Proceedings of the 10th International Workshop on Serverless Computing (Hong Kong, Hong Kong) (WoSC10 '24)*. 1–6.
 - [57] Shangguang Wang and Qing Li. 2023. Satellite Computing: Vision and Challenges. *IEEE Internet of Things Journal* 10, 24 (2023), 22514–22529. <https://doi.org/10.1109/JIOT.2023.3303346>
 - [58] Zhaojie Wen, Qiong Chen, Yipei Niu, Zhen Song, Quanfeng Deng, and Fangming Liu. 2024. Joint Optimization of Parallelism and Resource Configuration for Serverless Function Steps. *IEEE Transactions on Parallel and Distributed Systems* 35, 4 (2024), 560–576. <https://doi.org/10.1109/TPDS.2024.3365134>
 - [59] Ruolin Xing, Mengwei Xu, Ao Zhou, Qing Li, Yiran Zhang, Feng Qian, and Shangguang Wang. 2024. Deciphering the Enigma of Satellite Computing with COTS Devices: Measurement and Analysis. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (ACM MobiCom '24)*. Association for Computing Machinery, New York, NY, USA, 420–435. <https://doi.org/10.1145/3636534.3649371>
 - [60] Yanan Yang, Laiping Zhao, Yiming Li, Huan Yu Zhang, Jie Li, Mingyang Zhao, Xingzhen Chen, and Keqiu Li. 2022. INFless: a native serverless system for low-latency, high-throughput inference. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*. Association for Computing Machinery, 768–781. <https://doi.org/10.1145/3503222.3507709>
 - [61] Guangba Yu, Pengfei Chen, Zibin Zheng, Jingrun Zhang, Xiaoyun Li, and Zilong He. 2023. FaaS Deliver: Cost-Efficient and QoS-Aware Function Delivery in Computing Continuum. *IEEE Transactions on Services Computing* (2023), 1–16. <https://doi.org/10.1109/TSC.2023.3274769>
 - [62] Tetiana Zubko, Anshul Jindal, Mohak Chadha, and Michael Gerndt. 2022. MAFF: Self-adaptive Memory Optimization for Serverless Functions. In *Service-Oriented and Cloud Computing*, Fabrizio Montesi, George Angelos Papadopoulos, and Wolf Zimmermann (Eds.). Springer International Publishing, 137–154. https://doi.org/10.1007/978-3-031-04718-3_9